

Kick-off

Projeto	Formandos — gestão administrativa de formaturas e eventos
Marco do cronograma	01/09/2026 (Kick-off + Sprint Backlog + definição do MVP)
Versão	1.0 — elaborada em 26/09/2026
Equipe	Luanderson Arlindo, Luiz Orlando, José Renato e Vinícius
Repositório	https://github.com/luandersonarlindo/Formandos

1. Objetivo do kick-off

Alinhar a equipe sobre **o que será construído, por quê, por quem e até quando**, antes de escrever a primeira linha de código. Este documento registra o ponto de partida do projeto: problema, objetivo, escopo inicial, equipe, cronograma, forma de trabalho e riscos.

2. O problema

A organização de uma formatura reúne muita gente e muita decisão: local, comida, música, tarefas, fornecedores e dúvidas dos formandos. Hoje isso costuma se espalhar por grupos de mensagens, planilhas e conversas soltas. Consequências típicas:

- decisões tomadas por quem fala mais alto, e não pela preferência real da turma;
- dúvidas repetidas e respostas perdidas no meio das mensagens;
- tarefas sem responsável nem prazo claros;
- informações do evento (data, local, programação) desatualizadas ou difíceis de achar.

3. O produto em uma frase

Formandos é um aplicativo web onde a comissão organizadora e os formandos de uma turma decidem, perguntam e acompanham tudo o que envolve a formatura em um só lugar: votações por categoria com relatório, perguntas e respostas com votos, tarefas, programação e fornecedores.

4. Objetivos do projeto

1. Entregar um **MVP funcional** que a turma consiga usar de verdade (detalhado em *Definição do MVP*).
2. Aplicar em um produto real os conteúdos da disciplina: desenvolvimento web com React e Next.js, banco de dados relacional, autenticação, testes e trabalho em equipe com Git.
3. Documentar o projeto de forma que qualquer pessoa consiga entender, rodar e evoluir o código.

5. Escopo inicial (alto nível)

Área	O que entra
Acesso	Login por conta Google e por email e senha; entrada em uma turma por código de convite
Decisão	Enquetes por categoria, catálogo padrão pronto e catálogos personalizados, relatório com gráficos
Comunicação	Perguntas e respostas com votos dos colegas e moderação da comissão
Organização	Dashboard com contagem regressiva, programação, tarefas com responsável e prazo
Mercado	Vitrine de fornecedores cadastrados pela comissão
Administração	Gestão de membros, papéis e código de convite

Fora do escopo do primeiro ciclo: hospedagem em produção, aplicativo móvel nativo, pagamentos e notificações por email de eventos da turma.

6. Equipe

Integrante	Papel no projeto
Luanderson Arlindo	Desenvolvimento (responsável pelo repositório no GitHub)
Luiz Orlando	Desenvolvimento
José Renato	Desenvolvimento
Vinícius	Desenvolvimento

A atribuição de *Product Owner* e *Scrum Master* entre os quatro integrantes está no documento **Papéis**.

7. Cronograma macro

Datas do *Cronograma de Entregas* – 2026 da disciplina.

Data	Entrega
01/09/2026	Kick-off + Sprint Backlog + definição do MVP
08/09/2026	Termo de Abertura + visão do produto + papéis
15/09/2026	Stakeholders + requisitos prioritários
22/09/2026	Arquitetura inicial + organização do repositório Git
29/09/2026	Sprint 1: incremento funcional + documentação
06/10/2026	Fechamento da Sprint 1 + revisão dos artefatos
13/10/2026	Apresentação da Fábrica (AV I)

Data	Entrega
20/10/2026	Revisão + retrospectiva + planejamento da Sprint 2
27/10/2026	Status Report: código + documentação + qualidade
03/11/2026	Plano, casos e resultados de testes
10/11/2026	Sprint 2: integração, testes e estabilização
17/11/2026	Sprint Backlog + Status Report II
24/11/2026	Entrega final do produto + documentação + pré-banca

8. Forma de trabalho

- **Método:** Scrum adaptado a uma equipe pequena, com **duas sprints** (a Sprint 1 fecha em 06/10 e a Sprint 2 termina em 10/11), cada uma com planejamento, acompanhamento, revisão e retrospectiva.
- **Backlog:** o *Product Backlog* fica no documento **Sprint Backlog** e no repositório; cada item tem prioridade e estimativa relativa.
- **Código:** GitHub (`main` como ramo principal), commits pequenos no padrão *Conventional Commits* em português (`feat` , `fix` , `docs` , `test` , `chore`). Regras completas em **Organização do Repositório Git**.
- **Definição de pronto:** item concluído = funciona, tem validação de entrada e permissão, passa nos testes automatizados, foi visto no navegador e está documentado quando muda o comportamento.
- **Segredos:** chaves e senhas ficam só em `.env.local` , que nunca vai para o GitHub.

9. Decisões tomadas no início

#	Tema	Decisão
1	Autenticação	Better Auth (estável), no lugar do Auth.js, que segue em beta
2	Formas de login	Google (gratuito) e email e senha com confirmação por link
3	Banco de dados	PostgreSQL com SQL puro (driver <code>pg</code>), sem ORM, para aprender o SQL de verdade
4	Votos	Identificados: a comissão vê quem votou; o formando pode mudar o voto
5	Turmas	Um formando pertence a uma turma; quem é administrador pode participar de várias
6	Catálogos de enquetes	Um catálogo padrão de 8 categorias e catálogos personalizados por turma
7	Hospedagem	O projeto roda localmente e o código fica no GitHub

10. Riscos iniciais

Risco	Probabilidade	Impacto	Resposta
Equipe faz o primeiro projeto com Next.js e React	Alta	Médio	Guia de estudo (docs/guia-de-estudo.md) e leitura da documentação do Next.js instalada no projeto
Next.js 16 tem mudanças que quebram tutoriais antigos	Média	Médio	Regra no AGENTS.md : ler a documentação local antes de codar
Prazo curto entre as entregas	Alta	Alto	MVP enxuto e priorizado; itens extras vão para a Sprint 2
Login com Google depende de configuração externa	Média	Médio	Login por email e senha como alternativa
Vazamento de segredos no GitHub	Baixa	Alto	.gitignore para .env* e credenciais, e revisão antes de cada commit
Contribuição concentrada em poucas pessoas	Média	Médio	Dividir os itens da sprint entre os quatro integrantes e usar ramos e revisões

11. Próximos passos

1. Fechar o **Sprint Backlog** e a **Definição do MVP** (documentos desta mesma entrega).
2. Elaborar o **Termo de Abertura**, a **Visão do Produto** e os **Papéis** até 08/09.
3. Levantar **Stakeholders** e **Requisitos Prioritários** até 15/09.
4. Definir a **Arquitetura Inicial** e organizar o repositório Git até 22/09.